

## 3.8 Errors and Exceptions (Try-Except)

ایکسپشن (Exception) چلتے پروگرام کی غلطی ہے، جیسے غیر موجود فائل کھولنا۔

CS XII

Chapter 3: Programming Fundamentals

Lecture 16 of 17

### 3.8 Errors and Exceptions (Try-Except)

#### LEARN IT

An **exception** is an error raised while the program is running — for example opening a file that is not there. Without handling, Python prints a traceback and **stops**. A `try / except` block catches the error and lets you show a clear message instead of crashing.

| Error type                     | Meaning                                                        |
|--------------------------------|----------------------------------------------------------------|
| <code>FileNotFoundError</code> | The path does not exist.                                       |
| <code>PermissionError</code>   | The operating system will not let you read or write that file. |
| <code>IOError</code>           | A general input/output problem (disk full, disk disconnected). |

Put risky code in `try`. Put the recovery in `except ErrorType`. A final `except Exception as e` can catch anything else, using `e` as the error object.

```
try:
    with open('missing_data.txt', 'r') as f:
        content = f.read()
        print(content)
except FileNotFoundError:
    print("Error: The file does not exist. Please check the name.")
except Exception as e:
    print(f"An unexpected error occurred: {e}")
```

#### OUTPUT

Error: The file does not exist. Please check the name.

Without `try`, the same open raises:

#### OUTPUT

```
Traceback (most recent call last):
  File "missing.py", line 1, in <module>
    open('missing_data.txt', 'r')
FileNotFoundError: [Errno 2] No such file or directory: 'missing_data.txt'
```

#### WORKED EXAMPLE

When the file exists, the `except` blocks are skipped.

```
with open('ok.txt', 'w') as f:
    f.write("Present\n")

try:
    with open('ok.txt', 'r') as f:
        content = f.read()
        print(content, end="")
except FileNotFoundError:
    print("Error: The file does not exist. Please check the name.")
```

#### OUTPUT

Present

Professional software does not dump a red traceback on a user. If a file is missing or a disk fails, the user should see a calm sentence such as "Please check the file name" or "Please try again later."

#### REMEMBER IT

**try = attempt; except = backup plan.** Catch the **specific** error first (`FileNotFoundError`), then a general `Exception` if you need it. Order matters: a bare `except Exception` first would swallow the specific error.

#### COMMON MISTAKE

Do not wrap your entire program in one silent `except:` that ignores every mistake. You would hide real bugs. Catch the errors you expect, and keep the message honest. `'w'` and `'a'` create a missing file, so they do not raise `FileNotFoundError`; `'r'` does.

#### اردو میں سمجھیں

ایکسپشن (Exception) چلتے پروگرام کی غلطی ہے، جیسے غیر موجود فائل کھولنا۔ `try` میں خطرناک کوڈ لکھیں اور `except FileNotFoundError` میں صاف پیغام دکھائیں تاکہ پروگرام کریش نہ ہو۔ پیشہ ورانہ سافٹ ویئر صارف کو سرخ ٹریس بیک کی بجائے سمجھ آنے والا جملہ دکھاتا ہے۔

#### CHECK YOURSELF

1. Which exception is raised when `open('missing_data.txt', 'r')` cannot find the file?
2. Does the sample program crash, or print a message, when the file is missing?

Answers: 1. `FileNotFoundError`. 2. It prints "Error: The file does not exist. Please check the name." and continues.

## Extra board practice

These MCQs and questions are extra to the textbook. Close the notes, attempt them, then open the key.

### MCQ 1. try/except is for:

- a. Handling runtime errors without crashing the whole program
- b. Fixing syntax errors (missing colons)
- c. Faster Big O
- d. Drawing pie charts

### MCQ 2. FileNotFoundError is raised when:

- a. open r on a missing path
- b. 1/0
- c. int("a")
- d. A logic bug that prints 3 instead of 4

### MCQ 3. except ValueError catches:

- a. int("hi")
- b. Missing files
- c. SyntaxError of a colon
- d. Logic silent bugs

### MCQ 4. except Exception is:

- a. Broad — use with care
- b. The way to hide all bugs always
- c. A sort
- d. A key

### MCQ 5. else on try runs when:

- a. No exception occurred
- b. Always
- c. Only on errors
- d. On syntax errors

### MCQ 6. finally runs:

- a. Always (cleanup)
- b. Only on success
- c. Only on error
- d. Never with with

## Extra short questions

1. Why exceptions in file programs?
2. Show a try around int(input).
3. Does try fix logic errors?
4. Order of except clauses.

## Extra long question

1. Write a reader that tries to open marks.txt, handles missing file, skips bad lines with ValueError, and prints a count of good rows.

## Extra answer key

**MCQs:** 1. A 2. A 3. A 4. A 5. A 6. A

### Short

1. Missing files and bad lines are expected; crash is a poor UX.
2. try: n=int(input()) except ValueError: print("Enter a number").
3. No — wrong formula still wrong.
4. Specific first, broader later.

### Long

1. Full code. Sample: file missing message; file with 80, x, 90 → 2 good. Mention not using bare except.